

Task 02 - Part 01: Question Answers

C# Fundamentals - Variables, Types, Operators, Casting

Q1. What is the shortcut to comment and uncomment a selected block of code in Visual Studio?

In Visual Studio, select the block of code and press **Ctrl+K, Ctrl+C** to comment it out (adds // to the start of each selected line), and press **Ctrl+K, Ctrl+U** to uncomment it (removes the // from each selected line). For a single line, you can also just place the cursor on that line without selecting anything and use the same shortcuts.

Q2. Explain the difference between a runtime error and a logical error, with examples.

A **runtime error** occurs while the program is executing and causes it to crash or throw an exception; the code compiles fine, but something goes wrong during execution. Example: dividing an integer by zero (`int result = 10 / 0;`) throws a `DivideByZeroException`, or accessing an array index that doesn't exist throws an `IndexOutOfRangeException`.

A **logical error** occurs when the program runs without crashing, but produces an incorrect result because the logic/algorithm itself is wrong. Example: writing `'area = length + width;'` instead of `'area = length * width;'` - the program runs fine and gives an answer, but the answer is wrong because the formula used is incorrect.

Q3. Why is it important to follow naming conventions such as PascalCase in C#?

Naming conventions make code more readable, consistent, and maintainable, especially when working in a team. `PascalCase` (e.g., `CustomerName`, `CalculateTotal`) is the standard C# convention for class names, method names, and properties, while `camelCase` is typically used for local variables and parameters. Following these conventions helps other developers immediately recognize what kind of identifier they're looking at, makes code easier to review and debug, and aligns with the .NET framework's own naming style, so the code integrates smoothly with built-in libraries and other developers' code.

Q4. Explain the difference between value types and reference types in terms of memory allocation.

Value types (such as `int`, `double`, `bool`, `char`, and `struct`) store their actual data directly in the memory location where the variable is declared - typically on the **stack**. When a value type is assigned to another variable, a full copy of the data is made, so changes to one variable do not affect the other.

Reference types (such as class instances, arrays, and strings) store a reference (a memory address / pointer) on the stack, while the actual object data lives on the **heap**. When a reference type is assigned to another variable, only the reference (address) is copied - both variables end up pointing

to the same object in memory, so changing the object through one variable is visible through the other.

Q5. What will be the output of: `int a = 2, b = 7; Console.WriteLine(a % b);` Explain why.

The output is **2**. The `%` operator returns the remainder of integer division. Since 2 divided by 7 equals 0 with a remainder of 2 (because 7 goes into 2 zero times), `a % b` evaluates to 2. Whenever the left-hand operand (the dividend) is smaller than the right-hand operand (the divisor), the remainder is simply the dividend itself.

Q6. How does the `&&` (logical AND) operator differ from the `&` (bitwise AND) operator?

`&&` (logical AND) works on boolean expressions and uses **short-circuit evaluation**: if the first (left) condition is false, the second condition is never evaluated at all, since the overall result is already known to be false. This is useful for performance and for avoiding errors (e.g., checking that an object is not null before accessing its members).

`&` (bitwise AND) can be used both on booleans and on integer types. When used on integers, it performs a bit-by-bit AND operation on the binary representation of the numbers. When used on booleans, it behaves like a logical AND but **without short-circuiting** - both sides are always evaluated, even if the first one is already false.

Q7. Why is explicit casting required when converting a double to an int?

Explicit casting is required because converting a double to an int is a **narrowing conversion** that can lose data - specifically, the fractional (decimal) part of the number is discarded (truncated), and if the double's value is outside the range of int, an `OverflowException` can occur in a checked context. Because this conversion is potentially unsafe and lossy, C# does not allow it to happen implicitly; the developer must write an explicit cast, e.g., `(int)myDouble`, to consciously acknowledge that data may be lost.

Q8. What exception might occur if the input is invalid (when using `int.Parse`), and how can you handle it?

If the input string cannot be converted to a valid integer (e.g., it contains letters or symbols), `int.Parse` throws a **`FormatException`**. If the string is null, it throws an **`ArgumentNullException`**. If the number is valid but too large or too small to fit in an int, it throws an **`OverflowException`**.

These can be handled using a try/catch block around the `Parse` call, catching `FormatException` (and optionally `OverflowException`) and displaying a friendly error message instead of letting the program crash. Alternatively, **`int.TryParse`** can be used instead of `Parse` - it returns a bool indicating success or failure instead of throwing an exception, which avoids the need for try/catch entirely.

Q9. Given: `int x = 5; int y = ++x + x++;` What is the value of x after execution? Explain why.

After execution, **x = 7**.

Step by step: `++x` is the **prefix** increment, so x is incremented first (5 becomes 6), and 6 is used in the expression. Then `x++` is the **postfix** increment, so the CURRENT value of x (which is now 6) is used in the expression first, and x is incremented afterward (6 becomes 7). So $y = 6 + 6 = 12$, and after both operations x ends up as 7 (it was incremented twice: once by `++x` and once by `x++`).